

BRB NGX Analyst Platform — AWS Deployment Manual

Audience: Engineering / DevOps · **Status:** Canonical (supersedes the July 2025 `aws-go-live-runbook.md` and `deployment-guide.*`) · **Last updated:** 2026-08-22

This manual covers deploying the whole platform — the **NGX Analyst terminal**, the **Alternative Strategies engine** (with its background scheduler), the **public prospect portal**, and the **broker integration API** — as a single Next.js application on AWS.

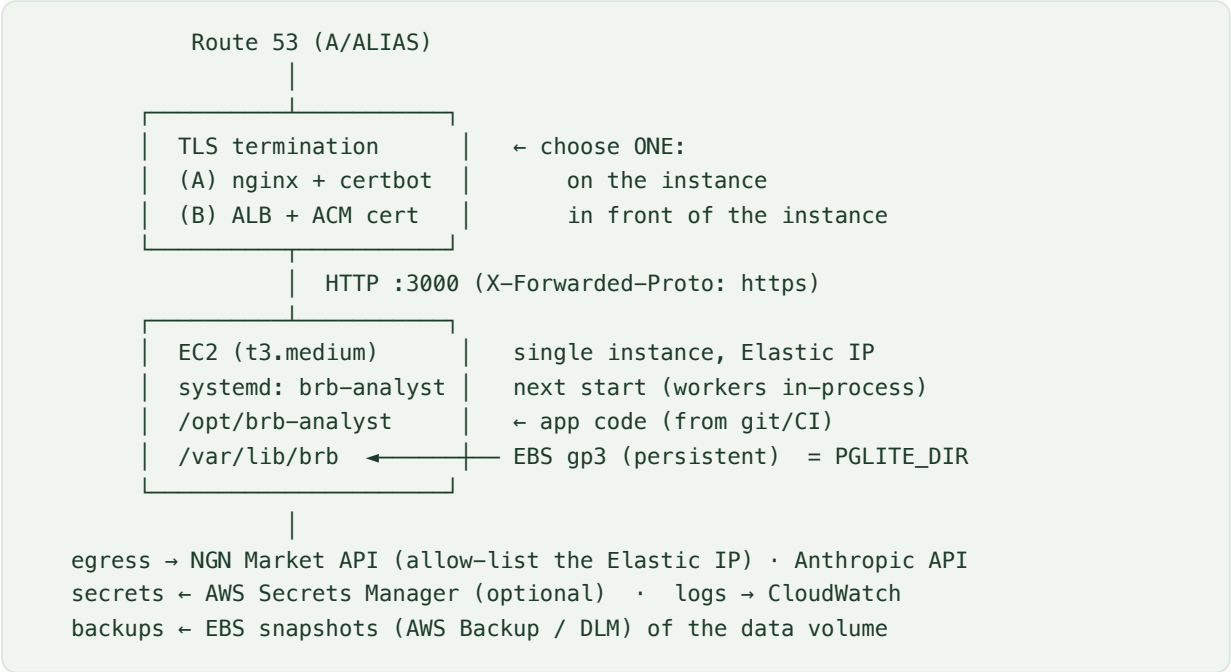
1. Architecture & why the topology is what it is

The application is a **single long-lived Node process** with three properties that dictate the deployment shape:

PROPERTY	WHERE	CONSEQUENCE FOR HOSTING
Embedded database — PGLite (Postgres compiled to WASM), stored as files under <code>PGLITE_DIR</code>	<code>src/lib/db/client.ts</code>	Needs a persistent block device . Only one process may open the directory at a time.
In-process schedulers — the alert worker and the engine's trading scheduler run on <code>setInterval</code> inside the server	<code>src/instrumentation.ts</code> → <code>alert-worker.ts</code> , <code>strategy-worker.ts</code>	Must run in a single always-on instance . Not compatible with request-scoped/scale-to-zero compute.
In-memory rate limiting — fixed-window counters in process memory	<code>src/lib/rate-limit.ts</code>	Correct only on one node . Multiple instances would each keep separate counters.

Therefore: deploy exactly one instance with a persistent disk. This platform is **not** a fit for Lambda, App Runner, scale-to-zero, or an autoscaled/multi-task service **as currently built**. The scaling ceiling and the migration path past it are documented in §11.

Recommended topology (primary)



Everything the app needs at rest lives on the **EBS data volume** (`/var/lib/brb`). Treat the EC2 instance as replaceable; treat the data volume (and `APP_ENCRYPTION_KEY`) as precious.

Repository deploy artifacts

FILE	PURPOSE
<code>.env.production.example</code>	Environment template — copy, fill, keep out of git
<code>deploy/brb-analyst.service</code>	systemd unit (single instance, workers in-process)
<code>deploy/nginx-brb-analyst.conf</code>	TLS reverse proxy (option A)
<code>deploy/render-env.sh</code>	Render <code>brb.env</code> from AWS Secrets Manager at boot (option)
<code>deploy/cloudwatch-agent.json</code>	Ship app + nginx logs and host metrics to CloudWatch
<code>scripts/qa.mjs</code> , <code>scripts/pentest.mjs</code>	Pre-deploy functional + security gates
<code>scripts/reset-admin.mjs</code>	Break-glass admin password reset

2. Prerequisites

- **AWS account** with permission to create EC2, EBS, Elastic IP, Security Groups, IAM roles, Route 53 records, ACM certs, Secrets Manager secrets, and (optionally) an ALB.
- A **domain** you control (e.g. `analyst.brbcapital.example`).
- **NGN Market** API key (`ngm_live_...`) and the ability to **allow-list the server's egress IP** in the NGN Market dashboard.
- **Anthropic (Claude)** API key (`sk-ant-...`).
- Toolchain on the build host: **Node 20 LTS** (the app is verified on `v20.x`), `git` , `openssl` .

3. Provision AWS resources

- **EC2 instance** — Ubuntu 22.04 LTS, `t3.medium` (**4 GB RAM**) **recommended** (a production `next build` and the AI/PGLite runtime are memory-hungry; `t3.small` works only if you build in CI and add swap — see §13). Place in a public subnet (option A) or a private subnet behind an ALB (option B).
- **Elastic IP** — allocate and associate. Gives a **stable egress IP** to allow-list with NGN Market and a stable inbound IP for DNS.
- **EBS data volume** — a **separate gp3 volume** (start at 20 GB), attached (e.g. `/dev/sdf`). Holds the database; kept apart from the root volume so it survives instance replacement and is snapshotted independently.
- **IAM instance role** — attach only what you use:
 - `secretsmanager:GetSecretValue` on your secret ARN (if using `render-env.sh`) - `CloudWatchAgentServerPolicy` (if shipping logs/metrics)
- **Security group (least privilege):**
 - Option A (nginx on box): `443/tcp` and `80/tcp` from `0.0.0.0/0`; `22/tcp` from your admin IP only.
 - Option B (ALB in front): instance allows `3000/tcp` **from the ALB security group only**; ALB allows `443 / 80` from the internet; `22/tcp` to the instance from admin IP only.
- **DNS** — Route 53 A record → Elastic IP (option A) or ALIAS → ALB (option B).

4. Base host setup

```
sudo apt update && sudo apt -y upgrade
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt -y install nodejs git
node -v      # expect v20.x

# Mount the EBS data volume (format ONCE — never mkfs an existing volume!)
lsblk                # find the device, e.g. nvme1n1
sudo mkfs -t ext4 /dev/nvme1n1      # ⚠ brand-new volume only
sudo mkdir -p /var/lib/brb
echo '/dev/nvme1n1 /var/lib/brb ext4 defaults,nofail 0 2' | sudo tee -a /etc/fstab
sudo mount -a

# Service user that owns the app + data
sudo useradd --system --create-home --shell /usr/sbin/nologin brb
sudo mkdir -p /var/lib/brb/pg && sudo chown -R brb:brb /var/lib/brb
```

`t3.small` only: add swap before building, or the build OOM-stalls (see §13.1).

5. Deploy the application code

Build **fresh source** — never copy a dev `.data/pg` or `.env.local`. The live DB seeds itself on first boot.

```
sudo mkdir -p /opt/brb-analyst && sudo chown brb:brb /opt/brb-analyst
sudo -u brb git clone <your-repo-url> /opt/brb-analyst # or rsync source, excluding .data
cd /opt/brb-analyst
sudo -u brb npm ci

# Run the quality gates here (or in CI – see §10) BEFORE building for prod:
sudo -u brb npm run typecheck
sudo -u brb npm test

# Production build. NEXT_PUBLIC_* values are inlined NOW, at build time – if you
# use the CAPTCHA, export NEXT_PUBLIC_TURNSTILE_SITE_KEY before this step.
sudo -u brb NODE_ENV=production npm run build
```

6. Secrets & environment

The app refuses to boot in production without `APP_ENCRYPTION_KEY` and `BRB_SEED_PASSWORD` — deliberate fail-safes (`src/lib/db/crypto.ts` , `src/lib/db/users.ts`).

Required: `NODE_ENV=production` , `APP_ENCRYPTION_KEY` , `BRB_SEED_PASSWORD` , `NGNMARKET_API_KEY` , `ANTHROPIC_API_KEY` , `PGLITE_DIR=/var/lib/brb/pg` . **Optional:** `PORT` , `NGNMARKET_BASE_URL` , member email (`APP_BASE_URL` , `SMTP_*` , `MAIL_FROM`), CAPTCHA (`TURNSTILE_SECRET_KEY` , `NEXT_PUBLIC_TURNSTILE_SITE_KEY`). Full list + notes in `.env.production.example` .

Option 6a — static env file

```
sudo mkdir -p /etc/brb
sudo cp /opt/brb-analyst/.env.production.example /etc/brb/brb.env
sudo nano /etc/brb/brb.env # fill EVERY required value
openssl rand -hex 32 # -> APP_ENCRYPTION_KEY (set ONCE, never change)
openssl rand -base64 24 # -> BRB_SEED_PASSWORD
sudo chown root:brb /etc/brb/brb.env && sudo chmod 640 /etc/brb/brb.env
```

Option 6b — AWS Secrets Manager (recommended for teams)

Store one secret (JSON object of the required keys), then render it at boot with the provided script:

```
aws secretsmanager create-secret --name brb/analyst/prod \
  --secret-string '{"APP_ENCRYPTION_KEY":"...","BRB_SEED_PASSWORD":"...","NGNMARKET_API_KEY":"..."}'
sudo apt -y install jq awscli
# render-env.sh runs as ExecStartPre in the systemd unit (below) and writes /etc/brb/brb.env
```

⚠ **APP_ENCRYPTION_KEY** is the master key for all stored secrets (broker API keys, in-app NGX/Anthropic keys). Generate it once, **escrow it securely, and never rotate it in place** — a restored data volume is undecryptable without the exact key that wrote it.

7. Run under systemd

```
sudo cp /opt/brb-analyst/deploy/brb-analyst.service /etc/systemd/system/  
sudo systemctl daemon-reload  
sudo systemctl enable --now brb-analyst  
sudo systemctl status brb-analyst  
journalctl -u brb-analyst -f      # watch for "[alerts] background worker started" and "[  
  
curl -s localhost:3000/api/health # {"ok":true,"status":"healthy",...}
```

Using Secrets Manager (6b)? Add a pre-start line to the unit (drop-in `systemctl edit brb-analyst`):

```
[Service]  
ExecStartPre=/opt/brb-analyst/deploy/render-env.sh
```

To ship app logs to CloudWatch as files, add a drop-in so systemd also writes a log file:

```
[Service]  
StandardOutput=append:/var/log/brb/app.log  
StandardError=append:/var/log/brb/app.log
```

(`sudo mkdir -p /var/log/brb && sudo chown brb:brb /var/log/brb` first.)

8. TLS — pick one

TLS is mandatory. In production the session cookie is issued `Secure` ; without HTTPS and an `X-Forwarded-Proto: https` header reaching the app, login silently fails.

Option A — nginx + certbot on the instance

```
sudo apt -y install nginx certbot python3-certbot-nginx  
sudo cp /opt/brb-analyst/deploy/nginx-brb-analyst.conf /etc/nginx/sites-available/brb-analyst  
sudo nano /etc/nginx/sites-available/brb-analyst      # set server_name  
sudo ln -s /etc/nginx/sites-available/brb-analyst /etc/nginx/sites-enabled/  
sudo rm -f /etc/nginx/sites-enabled/default  
sudo nginx -t  
sudo certbot --nginx -d analyst.brbcapital.example    # issues cert + auto-renew  
sudo systemctl reload nginx
```

The provided conf already forwards `X-Forwarded-Proto`, disables buffering for streamed AI responses, and sets a 320s read timeout (AI routes stream for up to 300s). Security headers are set by the **app tier** — nginx does not re-add them.

Option B — Application Load Balancer + ACM

- Request/validate an **ACM certificate** for the domain.
- ALB **HTTPS:443 listener** → target group → instance **:3000**; add an **HTTP:80 → HTTPS redirect**.
- Target group **health check**: `GET /api/health` (200 healthy, 503 when the DB is down).
- Set the target group **deregistration delay** and the ALB **idle timeout to ≥ 320s** so long AI streams aren't cut.
- HTTPS listeners forward `X-Forwarded-Proto: https` automatically — no nginx needed (you can run the app directly on :3000).

9. First-boot hardening (in the app)

1. Open `https://<domain>` → sign in as `admin@brb.local` with `BRB_SEED_PASSWORD`.
2. **Admin → Users & roles**: create real staff accounts (admin / PFM-pm / analyst) with strong passwords.
3. **Delete the seeded** `analyst@brb.local`, `pm@brb.local`, `admin@brb.local` (create your real admin first — the last admin cannot be removed).
4. Confirm **market data** loads (NGX key valid + Elastic IP allow-listed) and generate **one AI summary** (Anthropic key valid).
5. The **engine ships paused** and in **human-approval** mode — leave it so until you deliberately enable it (Engine → Settings). Broker accounts are created by an admin under Engine → Settings → Brokers.

10. Pre-deploy quality gates (CI)

Run these before promoting any build. All must pass; the app is verified green on all of them.

```
npm run typecheck          # 0 errors
npm test                   # 127/127 unit tests
NODE_ENV=production npm run build  # clean production build

# Dynamic checks against a running instance (staging or the box pre-cutover):
ADMIN_EMAIL=admin@brb.local ADMIN_PASSWORD='...' node scripts/qa.mjs          # 72 checks incl
ADMIN_EMAIL=admin@brb.local ADMIN_PASSWORD='...' node scripts/pentest.mjs    # active DAST, e
```

The broker-creation SQL fix (12 columns / 12 placeholders in `src/lib/db/brokers.ts`) **must** be in the deployed build — earlier builds 500 on broker creation on a fresh DB. `qa.mjs` 's deep loop covers it.

11. Scaling ceiling & migration path

The single-node design is deliberate and correct for launch scale. When you outgrow one box (throughput, HA/failover, or zero-downtime deploys), migrate in this order — none is required to go live:

1. **Database → RDS/Aurora Postgres.** Replace the PGLite client in `src/lib/db/client.ts` with a `pg` pool. The SQL is standard Postgres and schema creation is idempotent, so this is the highest-value, lowest-risk first move. Removes the single-writer disk constraint and enables multiple app instances.
2. **Rate limiting → Redis/ElastiCache.** Swap the in-memory `Map` in `src/lib/rate-limit.ts` for a shared store so limits hold across instances.
3. **Schedulers → a single owner.** Move the alert/engine workers out of every web instance into one dedicated worker service (or trigger via EventBridge Scheduler), so N web instances don't each fire the cadences. The engine already claims each job atomically (`strategy_runs UNIQUE(kind, run_key)`), which prevents double-fire, but you still want one intended runner.

Only after (1)–(3) can you run the web tier as an autoscaled ECS/Fargate service behind the ALB.

12. Backups & disaster recovery

- The entire database is `PGLITE_DIR` on the EBS volume. Enable **scheduled EBS snapshots** via **AWS Backup** or a **DLM** lifecycle policy (e.g. daily, 30-day retention).
- **Escrow** `APP_ENCRYPTION_KEY` in a separate secure location. A restored volume is useless without it.
- **Restore drill:** create a volume from a snapshot → attach to a fresh instance at `/var/lib/brb` → deploy code (§5) → same `APP_ENCRYPTION_KEY` in env → start. Practise this once before go-live.
- **RPO/RTO:** RPO = snapshot interval; RTO = instance launch + code deploy (minutes). Document your chosen interval.

13. Monitoring, logging & operations

- **Health check:** `GET /api/health` — 200 when the DB answers, 503 otherwise. Wire it to the ALB target group and/or an external uptime monitor.

- **Logs:** `journalctl -u brb-analyst` locally; ship to CloudWatch with `deploy/cloudwatch-agent.json` (app log file + nginx logs + mem/disk metrics).
- **Alarms (suggested):** CloudWatch alarms on `mem_used_percent`, `disk_used_percent` on `/var/lib/brb` (DB growth), instance `StatusCheckFailed`, and target-group `UnHealthyHostCount`.
- **In-app:** Admin → Quota & Audit shows NGN Market usage and the analyst audit trail.

13.1 Common issues

SYMPTOM	CAUSE	FIX
Login "works" then bounces to <code>/login</code>	Secure cookie dropped — no HTTPS or missing <code>X-Forwarded-Proto: https</code>	Terminate TLS; ensure the proxy/ALB forwards <code>X-Forwarded-Proto</code>
Boot error: <code>APP_ENCRYPTION_KEY</code> is not set / <code>BRB_SEED_PASSWORD</code> is not set	Missing required secret in production	Set them in <code>brb.env</code> / Secrets Manager
<code>502 / 503</code> at the proxy	App not running or DB wedged	<code>systemctl status brb-analyst ; curl :3000/api/health ; check journalctl</code>
Market data 401 / "No valid NGN Market API key"	Key wrong or egress IP not allow-listed	Verify <code>ngm_...</code> key; allow-list the Elastic IP in the NGN dashboard
AI features 503	Anthropic key missing/invalid	Set <code>ANTHROPIC_API_KEY</code> (or add it in Admin → Settings)
DB error about the directory being locked	A second process opened <code>PGLITE_DIR</code> (e.g. a script while the service runs)	PGLite is single-writer — stop the service before running <code>reset-admin.mjs</code> or any DB script
<code>next build</code> stalls at "Creating an optimized production build"	Out of memory (thrashing swap)	Use <code>t3.medium</code> +, or add swap and build with <code>NODE_OPTIONS=--max-old-space-size=4096</code> , or build in CI
Locked out of all admin accounts	—	Stop the service, run <code>RESET_EMAIL=you@brb... RESET_PASSWORD='...' RESET_ROLE=admin npm run reset-admin</code> , restart

14. Upgrades & rollback

```
# In CI first: npm run typecheck && npm test && NODE_ENV=production npm run build
aws ec2 create-snapshot --volume-id vol-XXXX --description "pre-deploy $(date +%F)" # sr
cd /opt/brb-analyst
sudo -u brb git pull
sudo -u brb npm ci
sudo -u brb NODE_ENV=production npm run build
sudo systemctl restart brb-analyst
curl -s localhost:3000/api/health
```

- **Schema changes are additive and idempotent** (`CREATE TABLE / ALTER ... ADD COLUMN IF NOT EXISTS`) and apply automatically on boot — no manual migration step.
- **Brief downtime is expected** on restart (single instance, single-writer DB). Communicate a short maintenance window; the restart is seconds.

- **Rollback:** check out the prior release tag → `npm ci && npm run build` → restart. If a bad build touched data, restore the pre-deploy volume snapshot.
-

15. Security posture (already in place)

- **Security headers at the app tier** (`next.config.mjs`): CSP (production drops `'unsafe-eval'`), HSTS (2-year, preload), `X-Frame-Options: DENY` , `nosniff` , Referrer-Policy, Permissions-Policy — hold regardless of the proxy.
 - **Secrets:** broker API keys stored **hashed + AES-256-GCM encrypted**; session tokens stored as SHA-256 (raw token only in the `httpOnly` cookie); three isolated identity systems (staff / member / broker).
 - **Rate limiting** on all logins, signup, and AI endpoints. **Audit logging** across AI and trading surfaces.
 - **Least-privilege SG**, admin-only SSH, secrets off-disk (Secrets Manager option), and the two production boot fail-safes.
 - Verified by `scripts/pentest.mjs` (0 vulnerabilities) — re-run it against the live environment as the final gate.
-

16. Go-live checklist

- ☐ EC2 (`t3.medium` +) up; Elastic IP associated; DNS points to it (or to the ALB)
 - ☐ EBS data volume mounted at `/var/lib/brb` ; `PGLITE_DIR=/var/lib/brb/pg`
 - ☐ Security group least-privilege (443/80 public or via ALB SG; 22 admin-IP only)
 - ☐ `brb.env` / Secrets Manager `APP_ENCRYPTION_KEY` generated, **escrowed** , and `BRB_SEED_PASSWORD` strong stable; complete;
 - ☐ `NEXT_PUBLIC_*` (if used) present at **build** time
 - ☐ `typecheck` + `test` + `build` all `qa.mjs` (72) `pentest.mjs` (0 vuln) pass production green; + against staging
 - ☐ Service `active (running)` ; `[alerts]` and `[engine]` workers `curl :3000/api/health` → logged; 200
 - ☐ TLS live; HTTP→HTTPS `X-Forwarded-Proto: https` reaches the app; login persists over redirect; HTTPS
 - ☐ NGN Market: Elastic IP allow-listed; market data + one AI summary confirmed
 - ☐ Seeded `@brb.local` accounts removed; real staff created; engine **paused** at confirmed launch
 - ☐ EBS snapshot schedule enabled; restore drill done `APP_ENCRYPTION_KEY` backed up once; separately
 - ☐ CloudWatch agent shipping logs/metrics; health check + alarms wired
-

*BRB Capital Group — internal engineering document. Pair with the in-app **User Manual** for feature/operations detail. Deploy artifacts live in `deploy/` and `.env.production.example`.*